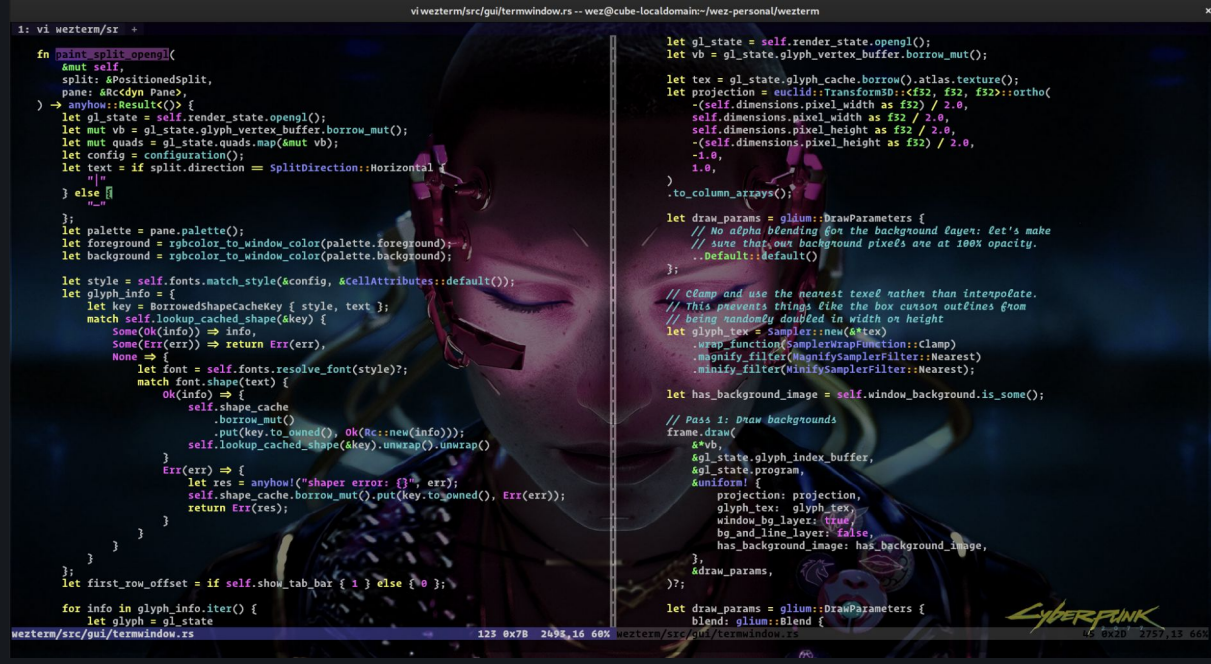


CISC 322/326 · Assignment 1 · October 2026

Conceptual Architecture of WezTerm

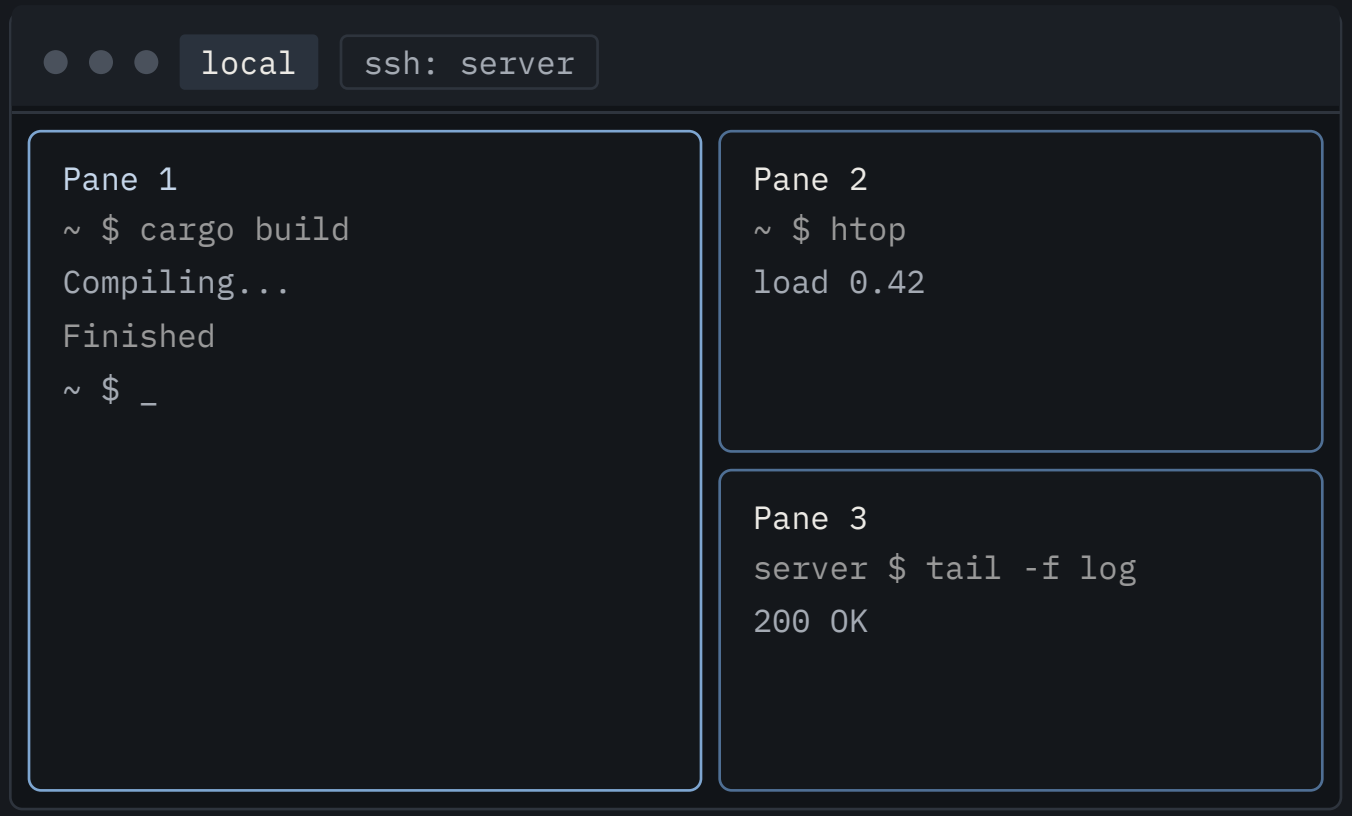
Arch Nemesis

CLICK HERE FOR PRESENTATION

The Team

Member	Email	Role	Main Report Sections
Paige Brossard	23mfq2@queensu.ca	Presenter	Use Cases, Diagrams, AI Collaboration Report
Sandy Khodak	23nr52@queensu.ca	Presenter	External interfaces, Conclusion
Kezia Sorensen	23hjl2@queensu.ca		Architecture
Ali Haider Cheema	20ahc1@queensu.ca	Team Lead	Architecture, abstract, Data dictionary, Naming conventions, Formatting, Lessons Learnt
Sienna Bragg	23dlc4@queensu.ca		Division of responsibilities, Evolution, Conclusion, Architecture
Stephanie Douglas	23pfc1@queensu.ca		Architecture, Diagrams, Introduction and Overview, Lessons Learnt

What is WezTerm?



Terminal emulator

+

Multiplexer

=

WezTerm

Derivation process

Since this is a conceptual architecture, we based it on WezTerm's public documentation, release notes, and GitHub discussions rather than the Rust source code.

We read through the documentation and grouped WezTerm's documented features and functions into different areas of responsibility.

We then traced key interactions, such as a keypress or splitting a pane, to understand how the different areas communicate with each other.

Five Subsystems

GUI/Input

Terminal Emulation

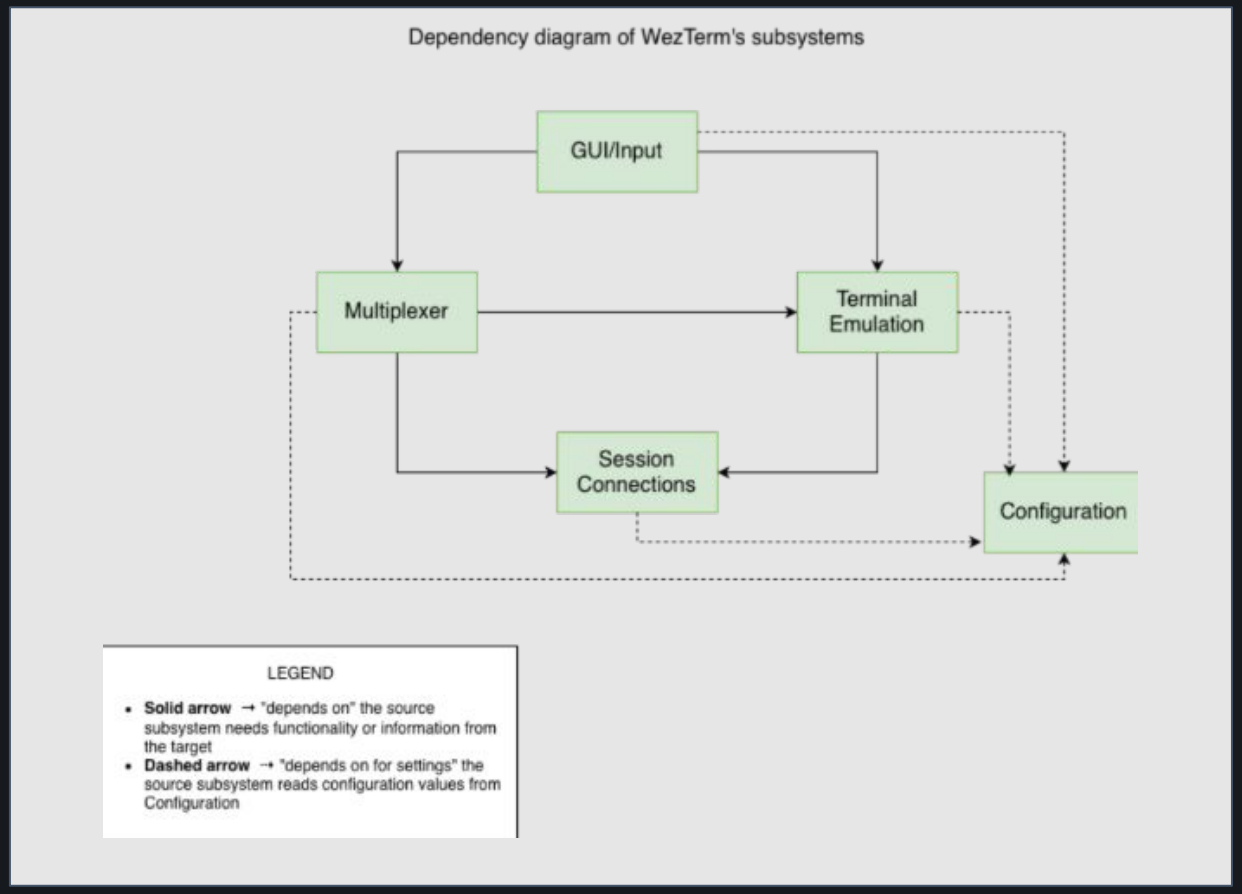
Configuration

Multiplexer

Session Connections

Sources [2] [3] [4] [12] 4

Dependencies



Legend

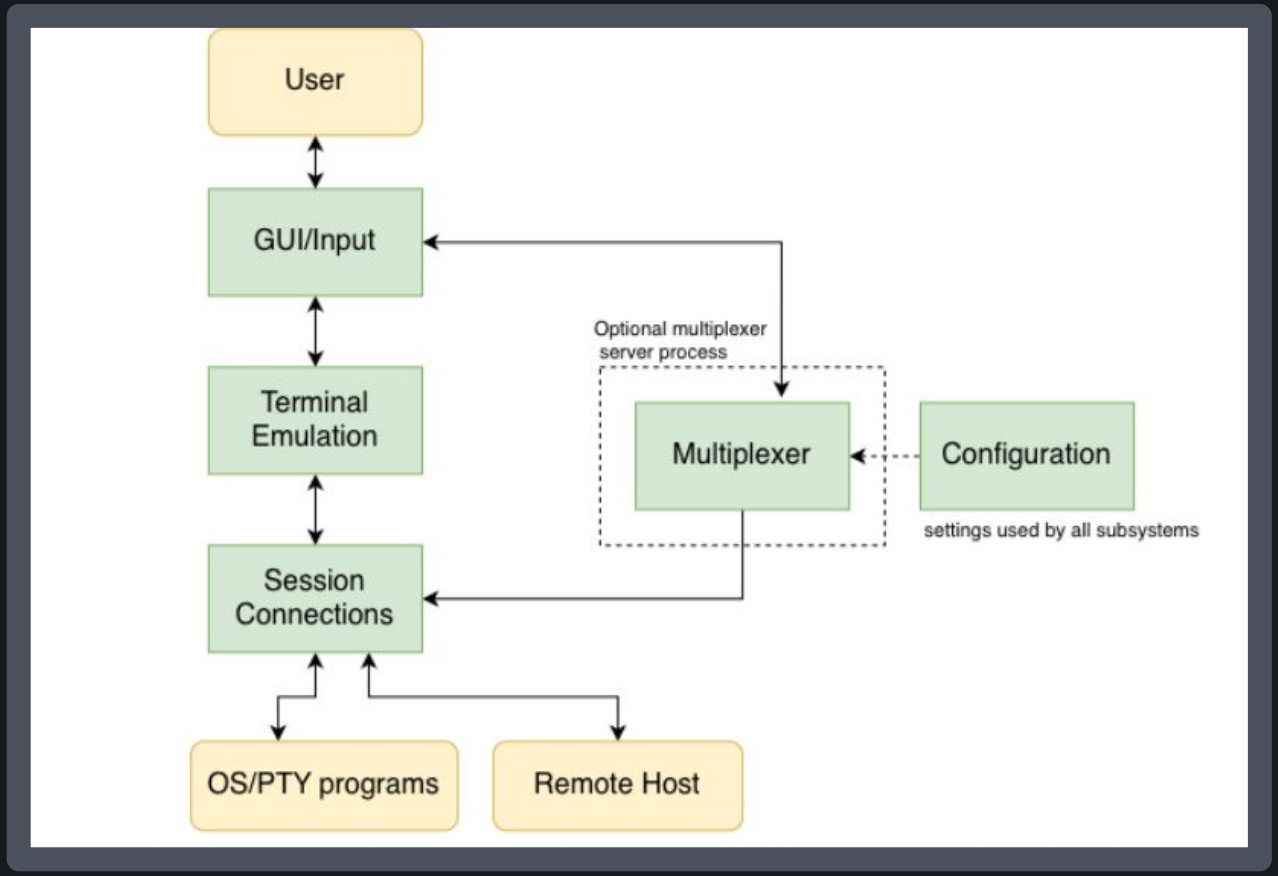
Conceptual subsystem

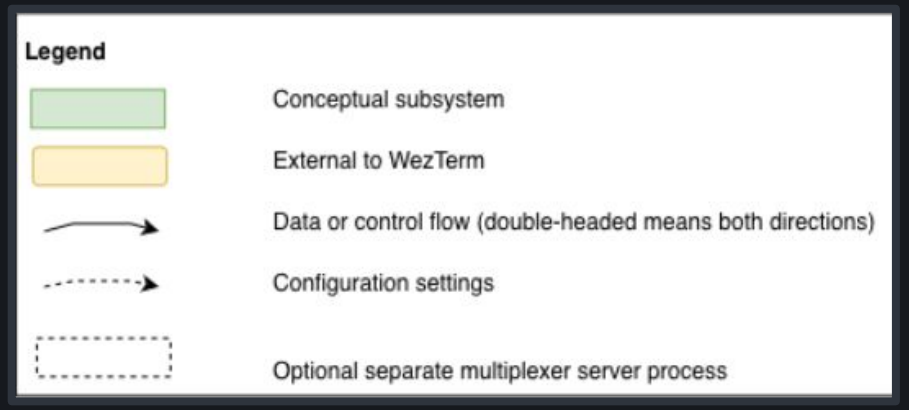
Depends on: needs function or info

Reads settings from Configuration

> Arrows show reliance, not data direction.

How The Data Moves





Architectural styles

Two styles shape WezTerm

Event-driven

- Work starts when an event arrives
- Many panes stay active at once
- Control flow is harder to trace

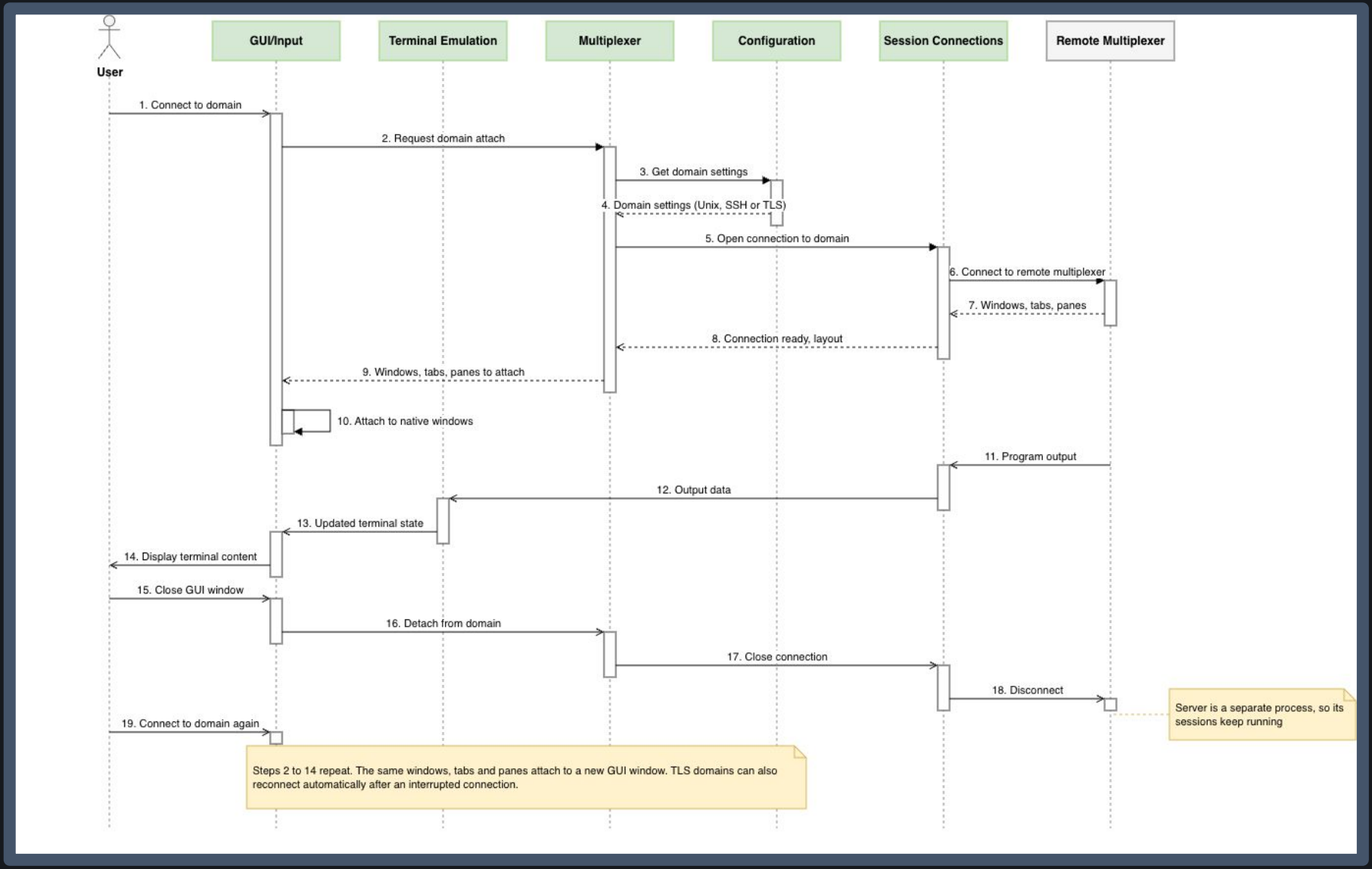
Client/Server

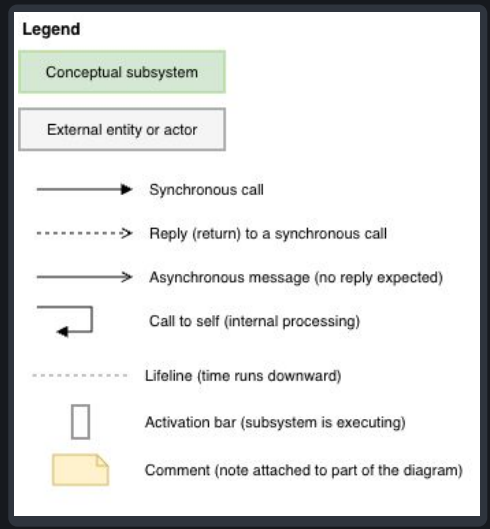
- GUI = client; multiplexer = separate server process
- Sessions survive after the GUI disconnects
- Latency and setup when the server is remote

Models we considered

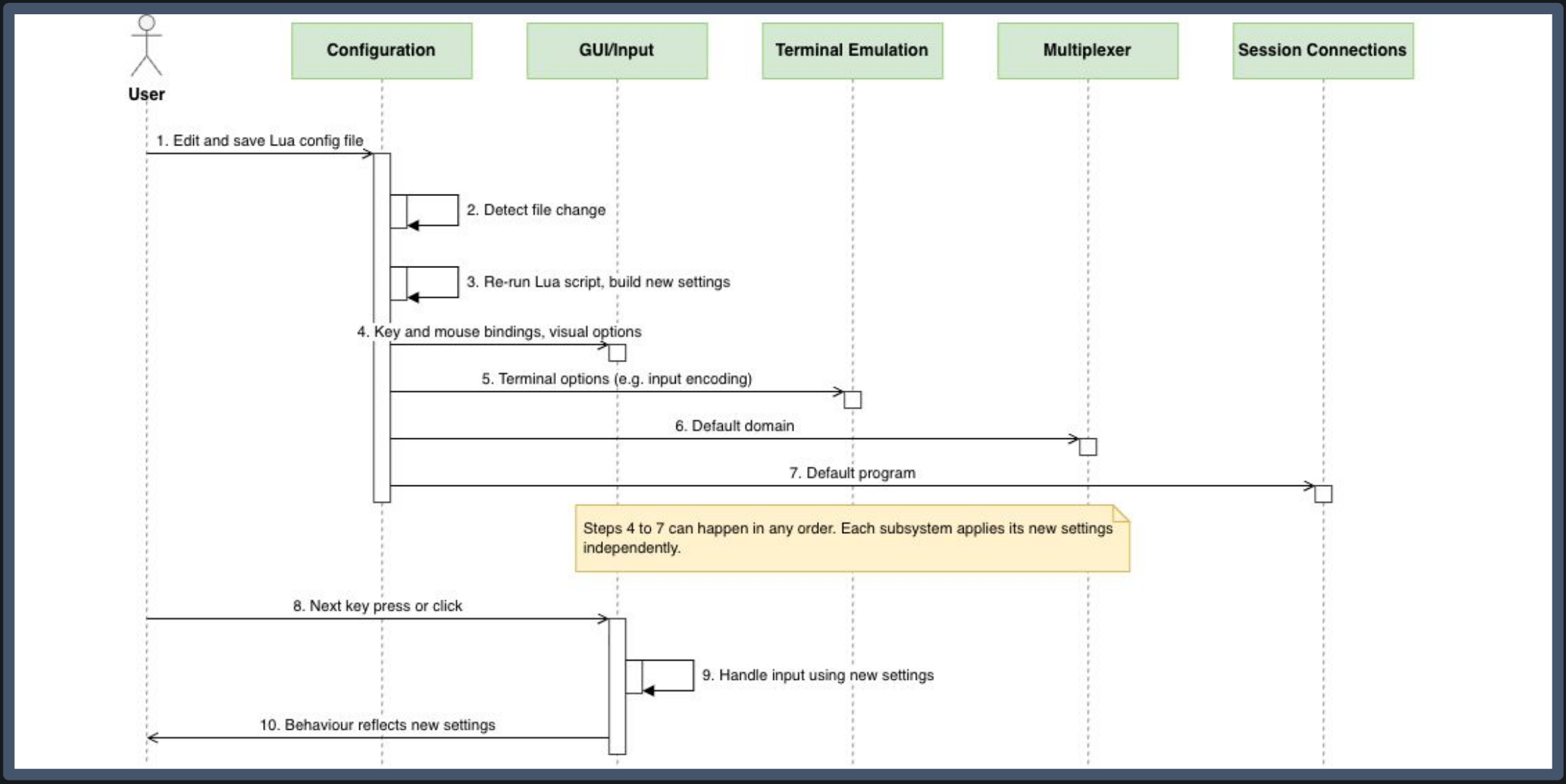
1. Merge Multiplexer and Terminal Emulation into one Pane
2. Configuration as a data path stage
3. Layered style

Use Case 1: Connection to a Remote Multiplexer

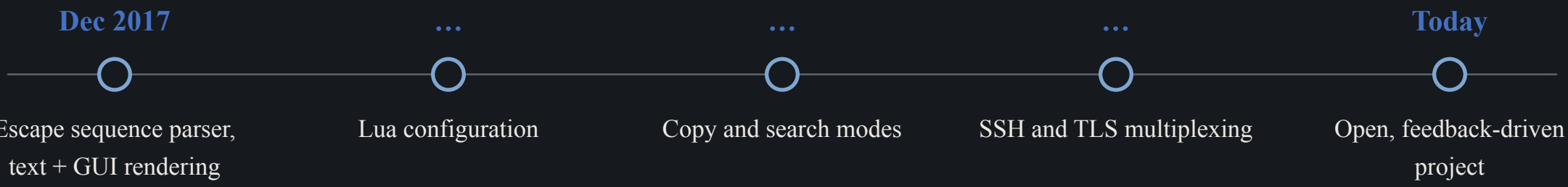




Use Case 2: Editing the Configuration



Evolution + division of labour

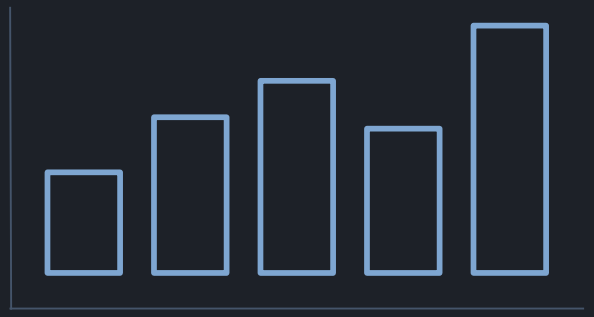


Limitations of the Model

From docs not code

Docs ≠ </>

Predicted, not measured



Processes, not threads

GUI
threads?

Mux server
threads?

Lessons learned

1

User docs make recovery difficult

2

Clear boundaries simplify the system

3

Integrated multiplexing increases maintenance effort

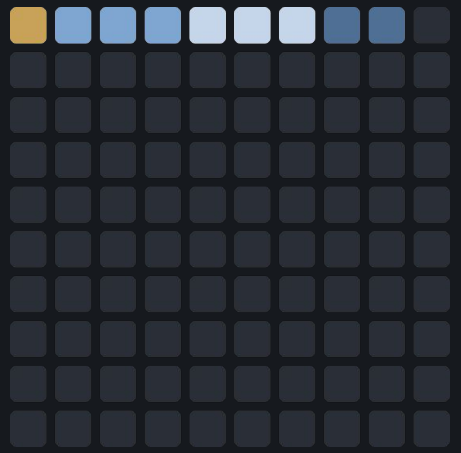
AI Teammates

ChatGPT-5.6 Luna

Research assistant

Claude Sonnet 5.5

Tutor and reviewer



~9%

of the final deliverable

- Text: 1%
- Diagrams: 3%
- Sources: 3%
- Review: 2%

Thank you

For more information, visit our website at:

https://archnemesis-26.github.io/ArchNemesis-Website/